



BURSA TEKNİK ÜNİVERSİTESİ

Bilgisayar Mühendisliği Bölümü

Node.js ile Web Programlama

DÖNEM PROJESİ TEKNİK RAPORU

Senaryo 2: Düşme ve Hareketsizlik Tespiti

Mobil Sensör Tabanlı Gerçek Zamanlı Güvenlik Platformu — CatchMe

Ders	Node.js ile Web Programlama
Senaryo	Senaryo 2 – Düşme ve Hareketsizlik Tespiti
Tarih	12 Haziran 2026
Danışman	Doç. Dr. İzzet Fatih ŞENTÜRK Arş. Gör. Yusuf KAYIPMAZ
Hazırlayanlar	Hasna Şahinoğlu Sudenur Elmas Halime Buse Yalçın

İçindekiler

1. Gereksinim Analizi ve Proje Tanımı
2. Kullanım Senaryosu
3. Sistem Mimarisi
4. Kullanılan Teknolojiler
5. Veri Modeli
6. Gerçekleştirilen Modüller
7. API Dokümantasyonu
8. Test Süreci
9. Karşılaşılan Kısıtlar ve Geliştirme Önerileri
10. Sonuç

1. Gereksinim Analizi ve Proje Tanımı

Günümüzde düşme olayları, özellikle yaşlı bireyler ve yalnız yaşayan kişiler için ciddi sağlık riskleri oluşturmaktadır. Acil müdahale süresinin kısaltılması, olayın otomatik tespitine ve yetkili kişilere anında bildirim yapılmasına bağlıdır. Bu çerçevede geliştirilen platform; akıllı telefonun donanımsal sensörlerini birer IoT uç düğümü olarak konumlandırmakta, toplanan ham sensör verilerini merkezi bir Node.js tabanlı web sunucusuna ileterek gerçek zamanlı analiz ve anomali tespiti gerçekleştirmektedir. Platform, makine öğrenmesi tabanlı düşme tespiti, hareketsizlik izleme, durum makinesi (state machine) tabanlı doğrulama mekanizması ve gerçek zamanlı web yönetim paneli ile uçtan uca çalışan bir güvenlik ekosistemi sunmaktadır.

1.1 Fonksiyonel Gereksinimler

Kullanıcılar sisteme kayıt olabilmeli, giriş yapabilmeli ve JWT tabanlı kimlik doğrulaması ile korunan kaynaklara erişebilmelidir.

Mobil uygulama, ivmeölçer ve jiroskop sensörlerinden 50 Hz frekansında veri toplamalı; bu verileri zaman damgasıyla birlikte sunucuya iletmelidir.

Backend, gelen sensör verilerini doğrulamalı, depolamalı ve düşme tespiti için analiz edebilmelidir.

Sistem, düşme tespiti durumunda AI modeli ve durum makinesi doğrulaması ile otomatik olarak yüksek öncelikli alarm kaydı oluşturmalıdır.

Kullanıcılar alarm geçmişini görüntüleyebilmeli ve alarmları çözüldü olarak işaretleyebilmelidir.

Her kullanıcı acil durum kişisi tanımlayabilmeli ve profil bilgilerini (profil tipi, uyku takvimi) yönetebilmelidir.

Sistem, kullanıcı hareketsizliğini izlemeli ve gece/gündüz dinamik eşiklerine göre hareketsizlik alarmı üretebilmelidir.

Admin rolüne sahip kullanıcılar, web paneli üzerinden tüm kullanıcıları, cihazları, düşme kayıtlarını, sensör grafiklerini ve özet istatistikleri gerçek zamanlı olarak görüntüleyebilmelidir.

1.2 Fonksiyonel Olmayan Gereksinimler

Sistem, Socket.IO aracılığıyla gerçek zamanlı veri akışını desteklemelidir.

Tüm şifreler bcryptjs ile hashlenerek saklanmalı; JWT token süresi kontrol altında tutulmalıdır.

API uç noktaları express-validator ile girdi doğrulamasına tabi tutulmalı; hatalı isteklere anlamlı HTTP hata yanıtları döndürülmelidir.

Uygulama helmet ve express-rate-limit middleware'leri ile temel güvenlik önlemlerine sahip olmalıdır.

AI mikroservisi 3 saniyelik zaman aşımı ile çağrılmalı; erişilemezse kural tabanlı fallback otomatik devreye girmelidir.

Redis, durum makinesi (fall state, inactivity state) verilerini TTL ile yönetmelidir.

Kritik işlevler Jest ve Supertest ile otomatik testlerle doğrulanmalı; kod kapsama oranı %90'ın üzerinde tutulmalıdır.

1.3 Proje Kapsamı

Proje, proje tanım belgesi kapsamında belirlenen zorunlu modüllerin tümünü içermektedir: mobil veri toplama, Node.js tabanlı arka uç, kullanıcı kimlik doğrulama ve yetkilendirme, veritabanı yönetimi, gerçek zamanlı veri akışı altyapısı, anomali/düşme tespiti ve alarm mekanizması. Bunların yanı sıra aşağıdaki ileri düzey özellikler de başarıyla gerçekleştirilmiştir:

- **AI Mikroservisi:** Python/FastAPI tabanlı makine öğrenmesi düşme tespiti servisi (scikit-learn modeli)
- **Web Yönetim Paneli:** React + Vite + Tailwind CSS ile gerçek zamanlı admin dashboard
- **Hareketsizlik Tespiti:** Redis tabanlı durum makinesi ile gece/gündüz dinamik eşikli hareketsizlik izleme
- **Düşme Durum Makinesi:** AI + varyans doğrulamalı çok aşamalı düşme onay mekanizması (NORMAL → IMPACT_DETECTED → FALL_CONFIRMED)
- **Çevrimdışı Veri Tamponlama:** Mobil uygulamada bellek içi offline kuyruk mekanizması
- **Jiroskop Entegrasyonu:** Jiroskop verisi hem AI modeline girdi olarak kullanılmakta hem de web panelinde görselleştirilmektedir

2. Kullanım Senaryosu

Geliştirilen platform, öncelikli olarak yaşlı bireyler, yalnız yaşayan kişiler ve bakım desteğine ihtiyaç duyan kullanıcılar için tasarlanmıştır. Aşağıda sistemin uçtan uca kullanım akışı açıklanmaktadır.

2.1 Birincil Kullanım Senaryosu: Düşme Tespiti ve Alarm

Kullanıcı, mobil uygulamayı açarak sisteme giriş yapar ve sensör izleme başlatılır. Uygulama arka planda çalışmaya devam ederek ivmeölçer ve jiroskop verilerini toplar. Her 75 örnekten oluşan yaklaşık 1.5 saniyelik sensör penceresi Socket.IO üzerinden backend'e iletilir. Backend, gelen pencereyi AI mikroservisine (FastAPI) yönlendirir. AI modeli, penceredeki ivmeölçer ve jiroskop verilerinden 6 istatistiksel öznitelik çıkararak düşme olasılığını hesaplar. Olasılık $\geq \%85$ eşiğini aştığında durum makinesi IMPACT_DETECTED aşamasına geçer; ardından düşük varyans doğrulaması ile düşme onaylanarak (FALL_CONFIRMED) yüksek öncelikli alarm kaydı oluşturulur. Kullanıcıya mobil uygulamada geri sayımlı düşme bildirimi gösterilir; "İyiyim" butonuyla alarmı iptal edebilir. AI servisine ulaşamadığında ise sistem otomatik olarak kural tabanlı (magnitude > 2.5 g) fallback mekanizmasına geçer.

2.2 İkincil Kullanım Senaryosu: Hareketsizlik Tespiti

Sistem, kullanıcının ivmeölçer varyansını sürekli olarak izler. Kullanıcının uyku takvimine göre dinamik eşikler belirlenir (gündüz: 2 saat, gece: 8 saat). Hareketsizlik süresi eşiği aştığında NORMAL → PRE_ALARM geçişi yapılarak kullanıcıya ön alarm gönderilir. Kullanıcı 60 saniye (ayarlanabilir) içinde "İyiyim, Ben Buradayım" butonuna basmazsa PRE_ALARM → CONFIRMED geçişi gerçekleşir ve acil durum alarmı oluşturulur.

2.3 Acil Durum Kişisi Yönetimi

Kullanıcı, profil sayfası üzerinden acil durum kişisi adı ve telefon numarasını tanımlayabilmektedir. Profil tipini (yaşlı, genç) seçerek AI modelinin yaş grubu parametresini etkileyebilir. Uyku takvimini (gece başlangıç-bitiş saatleri) ayarlayarak hareketsizlik eşiklerini kişiselleştirebilir.

2.4 Admin Senaryosu: Web Panel ile Gerçek Zamanlı İzleme

Admin rolüne sahip kullanıcılar, React tabanlı web paneline giriş yaparak merkezi bir dashboard üzerinden tüm sistemi izleyebilmektedir. Dashboard'da toplam kullanıcı, toplam alarm, bugünkü düşme ve çözülmemiş alarm sayıları anlık olarak gösterilir. Sensör grafiklerinde ivmeölçer büyüklüğü ve jiroskop büyüklüğü Recharts kütüphanesi ile zaman serisi olarak görselleştirilir. Cihaz listesi sayfasında tüm cihazlar çevrimiçi/çevrimdışı durumlarıyla birlikte izlenir. Alarm geçmişi sayfasında alarmlar filtrelenir ve çözüme kavuşturulur. Socket.IO üzerinden düşme ve hareketsizlik olayları panele anlık olarak iletilir.

3. Sistem Mimarisi

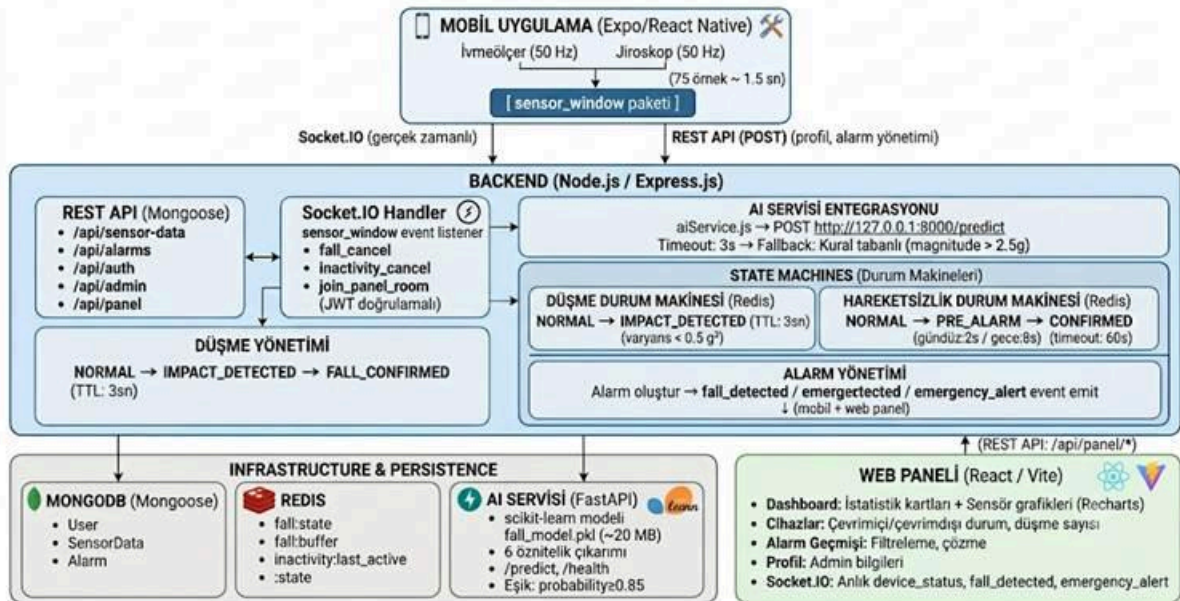
Platform, beş ana bileşenden oluşan mikroservis destekli katmanlı (layered) bir mimari üzerine inşa edilmiştir. Bu yaklaşım, her bileşenin bağımsız olarak geliştirilebilmesini, test edilebilmesini ve ölçeklendirilebilmesini sağlamaktadır.

3.1 Katman Yapısı

Katman	Teknoloji	Sorumluluk
Sunum Katmanı (Mobil)	React Native / Expo (TypeScript)	Sensör toplama, kullanıcı etkileşimi
Sunum Katmanı (Web)	React / Vite / Tailwind CSS	Admin dashboard, grafik görselleştirme
API ve İş Mantığı Katmanı	Node.js / Express.js	REST API, Socket.IO, durum makineleri
AI Katmanı	Python / FastAPI / scikit-learn	Makine öğrenmesi düşme tespiti
Veri Katmanı	MongoDB (Mongoose) + Redis	Kalıcı veri depolama + anlık durum yönetimi

3.2 Genel Sistem Akış Diyagramı

Genel Sistem Akış Diyagramı



3.3 Mobil Sensör Parametreleri

Parametre	Değer
Örnekleme Aralığı	20 ms (50 Hz)
Pencere Boyutu	75 örnek (~1.5 sn)
Sensörler	İvmeölçer + Jiroskop
İletim Protokolü	Socket.IO (WebSocket)
Çevrimdışı Kuyruk	Bellek içi, maks. 100 pencere
Reconnect sonrası	Otomatik kuyruk boşaltma

4. Kullanılan Teknolojiler

4.1 Backend

Teknoloji	Sürüm	Kullanım Alanı
Node.js	-	Sunucu çalışma zamanı
Express.js	5.2.x	HTTP sunucusu ve yönlendirme
Mongoose	9.6.x	MongoDB nesne modelleme (ODM)
Socket.IO	4.8.x	Gerçek zamanlı çift yönlü iletişim
jsonwebtoken	9.0.x	JWT oluşturma ve doğrulama
bcryptjs	3.0.x	Şifre hashleme
express-validator	7.3.x	Girdi doğrulama
helmet	8.2.x	HTTP güvenlik başlıkları
express-rate-limit	8.5.x	İstek hız sınırlama
axios	1.17.x	AI servisine HTTP istekleri
redis	6.0.x	Durum makinesi ve buffer yönetimi
dotenv	17.4.x	Ortam değişkeni yönetimi
cors	2.8.x	Cross-Origin Resource Sharing

4.2 AI Mikroservisi

Teknoloji	Kullanım Alanı
Python 3.12+	Programlama dili
FastAPI	ASGI web çerçevesi

uvicorn	ASGI sunucusu
scikit-learn	Makine öğrenmesi modeli (eğitim/tahmin)
numpy	Sayısal hesaplamalar
pandas	Veri çerçevesi oluşturma
pydantic	İstek/yanıt veri doğrulama

4.3 Mobil Uygulama

Teknoloji	Sürüm	Kullanım Alanı
React Native	0.81.x	Mobil uygulama çerçevesi
Expo	54.x	Geliştirme platformu
Expo Router	6.x	Dosya tabanlı navigasyon
expo-sensors	15.x	İvmeölçer ve jiroskop erişimi
expo-keep-awake	15.x	Ekran uyanık tutma
AsyncStorage	2.2.x	Yerel veri depolama (JWT kalıcılığı)
Socket.IO Client	4.8.x	Gerçek zamanlı sunucu iletişimi
TypeScript	5.9.x	Tip güvenli geliştirme
react-native-reanimated	4.1.x	Animasyonlar
react-native-gesture-handler	2.28.x	Dokunma hareketleri

4.4 Web Yönetim Paneli

Teknoloji	Sürüm	Kullanım Alanı
React	18.3.x	UI bileşen kütüphanesi
Vite	5.3.x	Hızlı geliştirme ve derleme aracı
Tailwind CSS	3.4.x	Utility-first CSS çerçevesi
React Router DOM	6.23.x	Sayfa yönlendirme
Recharts	2.12.x	Grafik ve veri görselleştirme
Axios	1.7.x	HTTP istemcisi
Socket.IO Client	4.7.x	Gerçek zamanlı panel güncellemeleri

4.5 Geliştirme ve Test

Teknoloji	Sürüm	Kullanım Alanı
Jest	30.x	Test çerçevesi
Supertest	7.2.x	HTTP test kütüphanesi
mongodb-memory-server	11.x	Bellek içi MongoDB (test ortamı)
nodemon	3.1.x	Otomatik yeniden başlatma (geliştirme)
ESLint	9.x	Kod kalite kontrolü (mobil)

5. Veri Modeli

Veri modeli, sistemin temel varlıklarını ve aralarındaki ilişkileri tanımlamaktadır. MongoDB'nin belge tabanlı yapısı, esnek şema tasarımına olanak tanırken Mongoose katmanı veri bütünlüğünü ve doğrulamayı güvence altına almaktadır. Redis ise anlık durum verilerini (düşme durumu, hareketsizlik durumu, sensör buffer'ı) TTL mekanizmasıyla yönetmektedir.

5.1 Varlık-İlişki Özeti

```
User —1:N—> SensorData
User —1:N—> Alarm
SensorData —1:1—> Alarm (sensorDataId referansı)
```

5.2 User Modeli

Alan	Tip	Zorunlu	Varsayılan	Açıklama
name	String	✓	-	Kullanıcı adı
email	String	✓ (unique)	-	E-posta adresi (küçük harf)
password	String	✓ (min 6)	-	Hashlenmiş şifre
role	String (enum)	-	"user"	"user" "admin"
profileType	String (enum)	-	"other"	"elderly" "worker" "athlete" "other"
emergencyContactName	String	-	""	Acil durum kişisi adı
emergencyContactPhone	String	-	""	Acil durum kişisi telefonu
sleepSchedule.nightStart	String	-	"23:00"	Gece başlangıç saati (HH:MM)
sleepSchedule.nightEnd	String	-	"07:00"	Gece bitiş saati (HH:MM)
createdAt	Date	oto	-	Mongoose timestamps
updatedAt	Date	oto	-	Mongoose timestamps

5.3 SensorData Modeli

Alan	Tip	Zorunlu	Varsayılan	Açıklama
userId	String	✓	-	Veri sahibi kullanıcı
deviceId	String	✓	-	Kaynak cihaz tanımlayıcı
timestamp	Date	-	Date.now	Ölçüm zaman damgası
accelerometer.x	Number	✓	-	İvme x eksen (g)
accelerometer.y	Number	✓	-	İvme y eksen (g)
accelerometer.z	Number	✓	-	İvme z eksen (g)
accelerometer.magnitude	Number	-	-	Bileşke ivme büyüklüğü ($\sqrt{x^2+y^2+z^2}$)
gyroscope.x	Number	✓	-	Açısal hız x eksen (rad/s)
gyroscope.y	Number	✓	-	Açısal hız y eksen (rad/s)
gyroscope.z	Number	✓	-	Açısal hız z eksen (rad/s)
isFallDetected	Boolean	-	false	Düşme tespit edildi mi
fallScore	Number	-	0	AI olasılık skoru veya magnitude
detectionMethod	String (enum)	-	"rule-based"	"rule-based" "ai-model"
createdAt	Date	oto	-	Mongoose timestamps
updatedAt	Date	oto	-	Mongoose timestamps

5.4 Alarm Modeli

Alan	Tip	Zorunlu	Varsayılan	Açıklama
userId	String	✓	-	Alarm sahibi kullanıcı
deviceId	String	✓	-	Alarmı tetikleyen cihaz
sensorDataId	ObjectId (ref)	✓	-	İlişkili SensorData referansı
alarmType	String (enum)	-	"fall"	"fall" "inactivity" "anomaly"
severity	String (enum)	-	"high"	"low" "medium" "high"
message	String	-	"Fall detected"	Alarm detay mesajı

isResolved	Boolean	-	false	Çözüldü mü
resolvedAt	Date	-	null	Çözüm zaman damgası
createdAt	Date	oto	-	Mongoose timestamps
updatedAt	Date	oto	-	Mongoose timestamps

5.5 Redis Durum Anahtarları

Anahtar Deseni	Değer	TTL	Açıklama
fall:state:{deviceId}	JSON {state, timestamp}	3 sn	Düşme durum makinesi (IMPACT_DETECTED)
fall:buffer:{deviceId}	List (float)	3 sn	Sensör varyans buffer'ı
inactivity:last_active:{deviceId}	ISO string	-	Son hareket zaman damgası
inactivity:state:{deviceId}	JSON {state, pre_alarm_start}	-	Hareketsizlik durum makinesi

6. Gerçekleştirilen Modüller

6.1 Kullanıcı Doğrulama ve Yetkilendirme Modülü

Sistem, JWT tabanlı durumsuz (stateless) kimlik doğrulama mekanizması ile güvence altına alınmıştır. Kayıt sırasında kullanıcı şifreleri bcryptjs kullanılarak hashlenmekte, giriş işlemi sonrasında oluşturulan JWT token ise sonraki isteklerde Authorization: Bearer başlığı aracılığıyla iletilmektedir. JWT token'ı mobil uygulamada AsyncStorage ile kalıcı olarak saklanmakta, böylece uygulama yeniden açıldığında oturum sürekliliği sağlanmaktadır.

Sistemde iki farklı kullanıcı rolü bulunmaktadır:

user: Kendi sensör verilerini, alarm kayıtlarını ve profil bilgilerini yönetebilir.

admin: Tüm kullanıcıları, cihazları, düşme kayıtlarını, sensör grafiklerini ve sistem istatistiklerini web paneli üzerinden görüntüleyebilir; yönetimsel işlemleri gerçekleştirebilir.

Admin rolüne özel uç noktalar adminOnly middleware'i ile korunmaktadır. Web panelinde ek olarak frontend tarafında da rol kontrolü yapılmaktadır; admin olmayan bir token ile giriş denendiğinde kullanıcı otomatik olarak çıkış yapılarak login sayfasına yönlendirilmektedir.

6.2 Sensör Veri Toplama ve İletim Modülü

Mobil uygulama, Expo Sensors API'si aracılığıyla ivmeölçer ve jiroskop sensörlerine 20 ms örnekleme aralığıyla erişmektedir. Her 75 örnekten oluşan yaklaşık 1.5 saniyelik veri penceresi, aşağıdaki yapıda sensor_window eventi olarak Socket.IO bağlantısı üzerinden backend'e iletilmektedir:

```
{
  "userId": "string",
  "deviceId": "string",
  "windowStart": "ISO 8601",
  "windowEnd": "ISO 8601",
  "sampleRateHz": 50,
  "readings": [
    {
      "timestamp": "ISO 8601",
      "accelerometer": { "x": 0.0, "y": 0.0, "z": 0.0 },
      "gyroscope": { "x": 0.0, "y": 0.0, "z": 0.0 }
    }
  ]
}
```

Bağlantı kesilmesi durumunda veriler bellek içi bir kuyruğa (maksimum 100 pencere) alınmakta; bağlantı yeniden kurulduğunda kuyruk otomatik olarak boşaltılmaktadır. Kuyruk dolduğunda en eski pencere çıkarılarak yeni pencere eklenmektedir (FIFO). Bu mekanizma, kısa süreli ağ kesintilerinde veri kaybını önlemektedir. Bağlantı durumu ve kuyruk boyutu, listener pattern ile dinlenebilmekte ve kullanıcı arayüzünde anlık olarak gösterilmektedir.

6.3 Düşme Tespit (AI Destekli Anomali Tespiti) Modülü

Düşme tespiti, iki katmanlı bir stratejiyle gerçekleştirilmektedir:

6.3.1 Birincil Yöntem: AI Modeli (Makine Öğrenmesi)

AI mikroservisi, Python/FastAPI üzerine inşa edilmiş bağımsız bir servistir. Node.js backend'den aiService.js modülü aracılığıyla HTTP POST isteği ile çağrılır. AI servisinde Random Forest (Rassal Orman) sınıflandırıcı modeli kullanılmıştır.

Öznitelik Çıkarım Süreci:

Gelen sensör penceresi üzerinden aşağıdaki 6 istatistiksel öznitelik hesaplanır:

Öznitelik	Hesaplama	Açıklama
-----------	-----------	----------

Yaş_Grubu	Kullanıcı profil tipinden türetilir	1 = yaşlı ("elderly"), 0 = diğer
Acc_Min	min(SMV_acc)	İvmeölçer SMV minimum değeri
Acc_Max	max(SMV_acc)	İvmeölçer SMV maksimum değeri
Acc_Var	var(SMV_acc)	İvmeölçer SMV varyansı
Gyro_Max	max(SMV_gyro)	Jiroskop SMV maksimum değeri (°/s)
Gyro_Var	var(SMV_gyro)	Jiroskop SMV varyansı (°/s)

SMV (Sinyal Büyüklük Vektörü) = $\sqrt{(x^2 + y^2 + z^2)}$

Not: Jiroskop verileri Expo sensörlerinden rad/s biriminde alınır ve model eğitiminde derece/s kullanıldığından 57.2958 çarpanıyla dönüştürülür.

Tahmin ve Eşikleme:

```
prediction = model.predict(features) → 0 (düşme yok) | 1 (düşme)
probability = model.predict_proba(features) → düşme sınıfı olasılığı (0.0 – 1.0)
```

```
is_fall = (prediction == 1) AND (probability ≥ 0.85)
```

FALL_PROBABILITY_THRESHOLD = 0.85 eşiği, hem modelin düşme tahmin etmesini (prediction=1) hem de bunu yüksek güvenle yapmasını (probability ≥ %85) gerektirir. Bu çift koşul, yanlış pozitif oranını önemli ölçüde düşürmektedir.

6.3.2 İkincil Yöntem: Kural Tabanlı Fallback

AI servisine 3 saniye içinde ulaşamadığında veya hata oluştuğunda, kural tabanlı fallback otomatik olarak devreye girer:

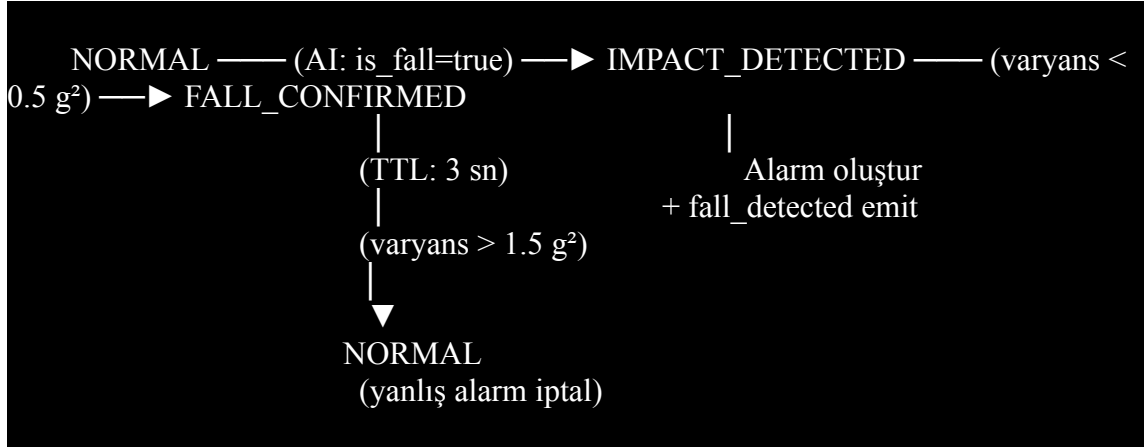
```
magnitude =  $\sqrt{(accX^2 + accY^2 + accZ^2)}$ 
FALL_THRESHOLD = 2.5 g

magnitude > FALL_THRESHOLD → isFallDetected = true
detectionMethod = "rule-based"
Alarm oluştur: { alarmType: "fall", severity: "high" }
```

Bu yaklaşım, AI servisinin erişilemez olduğu durumlarda bile sistemin işlevselliğini sürdürmesini garanti altına alır.

6.3.3 Düşme Durum Makinesi (Fall State Machine)

AI modelinin `is_fall = true` üretmesi tek başına alarm oluşturmaz. Yanlış pozitifleri azaltmak için Redis tabanlı bir durum makinesi uygulanmıştır:



Durum Geçişleri:

- NORMAL → IMPACT_DETECTED:** AI modeli düşme tespit ettiğinde geçiş yapılır. Redis'e 3 saniyelik TTL ile kaydedilir.
- IMPACT_DETECTED → FALL_CONFIRMED:** Sonraki sensör pencerelerinde ivme varyansı 0.5 g^2 altında kalıyorsa (kişi hareketsiz → düşmüş ve yerde kalmış) düşme onaylanır, alarm oluşturulur.
- IMPACT_DETECTED → NORMAL:** Varyans 1.5 g^2 üzerindeyse (kişi hareket ediyor → yanlış alarm) durum sıfırlanır.
- TTL Süresi Dolması:** 3 saniye içinde doğrulama gelmezse Redis otomatik olarak kaydeder, durum **NORMAL**'e döner.

6.4 Hareketsizlik Tespiti Modülü

Hareketsizlik tespiti, `inactivityManager.js` servisi tarafından Redis tabanlı bir durum makinesi ile gerçekleştirilmektedir. Her sensör penceresinde ivme varyansı hesaplanır ve hareket durumu değerlendirilir.

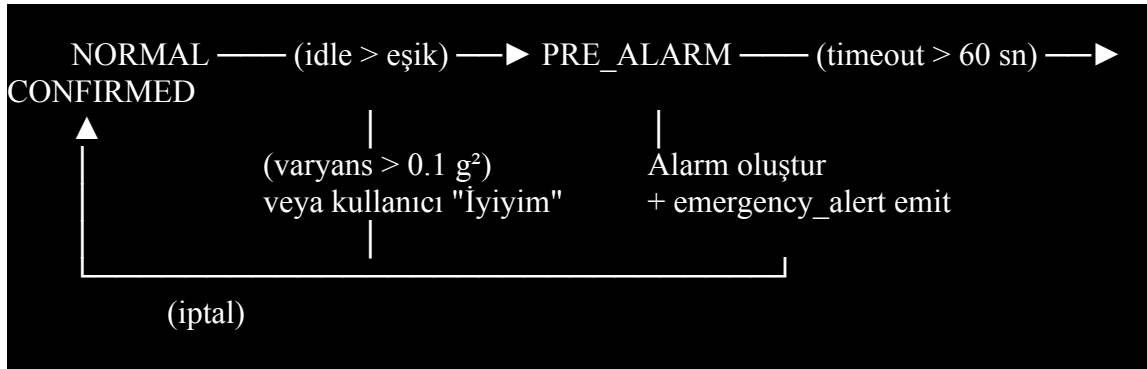
Dinamik Eşik Sistemi:

Kullanıcının `sleepSchedule` ayarına göre gece/gündüz dinamik eşikleri uygulanır:

Zaman Dilimi	Hareketsizlik Eşiği	Açıklama
Gündüz	2 saat (7200 sn)	Normal aktivite döneminde uzun hareketsizlik anormal
Gece	8 saat (28800 sn)	Uyku süresi boyunca hareketsizlik normal kabul edilir

Gece/gündüz ayrımı, kullanıcının uyku takvimindeki nightStart ve nightEnd saatlerine göre yapılır. Gece yarısını aşan aralıklar (örn: 23:00 → 07:00) desteklenmektedir.

Durum Makinesi:



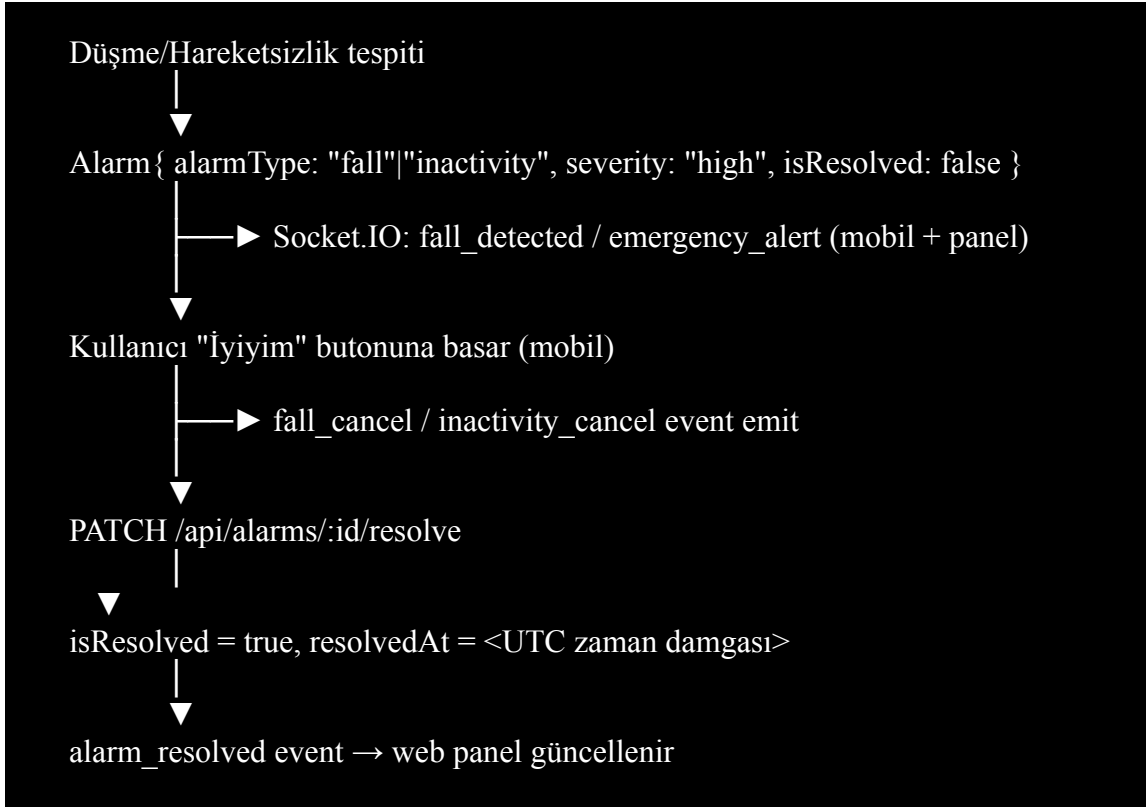
Durum Geçişleri:

- NORMAL → PRE_ALARM:** Hareketsizlik süresi dinamik eşiği aştığında. Kullanıcıya inactivity_pre_alarm eventi ile geri sayımlı bildirim gönderilir.
- PRE_ALARM → CONFIRMED:** 60 saniye (ayarlanabilir) içinde kullanıcıdan yanıt gelmezse. alarmType: "inactivity" ile alarm oluşturulur ve emergency_alert eventi emit edilir.
- PRE_ALARM → NORMAL:** Hareket tespit edilirse $(\text{varyans} > 0.1 \text{ g}^2)$ veya kullanıcı inactivity_cancel eventi gönderirse otomatik iptal.

Hareket Tespiti: sensorAnalyzer.js modülü, Redis'te tuttuğu kayan pencere buffer'ına her sensör verisi geldiğinde ivme büyüklüğü ekler (TTL: 3 sn) ve popülasyon varyansını hesaplar. Varyans $> 0.1 \text{ g}^2$ ise hareket var kabul edilir.

6.5 Alarm ve Bildirim Modülü

Düşme onayı (FALL_CONFIRMED) veya hareketsizlik onayı (CONFIRMED) gerçekleştiğinde sistem otomatik olarak bir Alarm kaydı oluşturmaktadır. Alarm yaşam döngüsü aşağıdaki gibi işlemektedir:



Mobil uygulamada düşme tespiti sonrası geri sayımlı (10 sn) bir alarm ekranı gösterilir. Kullanıcı süre dolmadan "İyiym (İptal Et)" butonuna basabilir.

6.6 Socket.IO Gerçek Zamanlı İletişim Modülü

Socket.IO bağlantıları JWT token ile doğrulanmaktadır. Token, handshake.auth.token veya Authorization: Bearer başlığından alınmaktadır. Kimlik doğrulama başarılı olduğunda kullanıcı bilgileri (profil tipi, uyku takvimi, rol) veritabanından çekilerek socket nesnesinde cache'lenir; böylece her sensör penceresinde tekrar DB sorgusu yapılmaz.

Oda Yönetimi:

user:{userId} — Her kullanıcı kendi odasına katılır (mobil bildirimleri alır).

panel:{userId} — Admin kullanıcılar otomatik olarak panel odasına katılır (web panel güncellemeleri alır).

Socket.IO Event'leri:

Event	Yön	Açıklama
sensor_window	Mobil → Backend	Sensör penceresi verisi

fall_detected	Backend → Mobil + Panel	Düşme tespit/onay bildirimi
fall_cancel	Mobil → Backend	Kullanıcı düşme alarmını iptal etti
alarm_resolved	Backend → Panel	Alarm çözüldü bildirimi
inactivity_pre_alarm	Backend → Mobil	Hareketsizlik ön alarm (geri sayım)
inactivity_cancelled	Backend → Mobil	Hareketsizlik ön alarm otomatik iptal
inactivity_cancel	Mobil → Backend	Kullanıcı hareketsizlik alarmını iptal etti
emergency_alert	Backend → Mobil + Panel	Acil durum alarmı (hareketsizlik onaylı)
device_status	Backend → Panel	Cihaz anlık durum (magnitude, gyro magnitude)
join_panel_room	Panel → Backend	Web panel oda katılım (opsiyonel, admin auto-join mevcut)

6.7 Web Yönetim Paneli (Admin Dashboard) Modülü

Admin dashboard, React 18 + Vite + Tailwind CSS ile geliştirilmiş tek sayfa uygulamasıdır (SPA). ProtectedRoute bileşeni ile admin rolü kontrolü yapılmaktadır. Panel, beş ana sayfadan oluşmaktadır:

6.7.1 Giriş Sayfası (LoginPage)

Admin kullanıcıların JWT token ile kimlik doğrulaması yaptığı sayfa. Token ve kullanıcı bilgileri AuthContext ile uygulamanın her yerinde erişilebilir kılınır. Admin olmayan kullanıcılar frontend tarafında da engellenir.

6.7.2 Dashboard Sayfası (DashboardPage)

Sistemin genel durumunu özetleyen ana sayfa. Üst kısımda istatistik kartları gösterilir:

Toplam Kullanıcı: Sistemdeki kayıtlı kullanıcı sayısı

Toplam Alarm: Oluşturulan toplam alarm sayısı

Bugünkü Düşmeler: Bugün tespit edilen düşme olayları

Alt kısımda SensorChart bileşeni ile gerçek zamanlı sensör grafikleri görselleştirilir. RecentAlarms bileşeni son 10 alarmı listeler.

6.7.3 Sensör Grafikleri (SensorChart)

Recharts kütüphanesi ile uygulanan zaman serisi grafikleri:

- **İvmeölçer Büyüklüğü:** accelerometer.magnitude değerlerinin zaman içindeki değişimi
- **Jiroskop Büyüklüğü:** gyroscopeMagnitude değerlerinin zaman içindeki değişimi (backend tarafından önceden hesaplanır)
- Düşme tespit edilen noktalar grafik üzerinde işaretlenir
- 1–24 saat arası zaman aralığı seçilebilir
- Cihaz ve kullanıcıya göre filtreleme yapılabilir
- Socket.IO üzerinden gelen device_status event'leri ile grafik anlık olarak güncellenir

6.7.4 Cihaz Listesi Sayfası (DevicesPage)

Sistemdeki tüm cihazları listeleyen sayfa:

- Cihaz ID, sahip kullanıcı, son görülme zamanı
- Çevrimiçi/çevrimdışı durumu (son 5 dakika içinde veri geldiyse çevrimiçi)
- Toplam düşme sayısı
- Son ivme büyüklüğü değeri

6.7.5 Alarm Geçmişi Sayfası (AlarmHistoryPage)

- Tüm alarm kayıtlarının listelendiği ve yönetildiği sayfa:
 - Alarm tipi (düşme/hareketsizlik), ciddiyeti, durumu
 - Alarmlar detay mesajıyla birlikte gösterilir
 - İlişkili sensör verisi referansı ile birlikte popüle edilir

6.7.6 Profil Sayfası (ProfilePage)

Admin kullanıcının kendi profil bilgilerini görüntülediği ve güncelleme yapabildiği sayfa.

6.7.7 Panel REST API Uç Noktaları

Web paneline özel olarak /api/panel öneki altında dört uç nokta tanımlanmıştır (tümü protect + adminOnly middleware'i ile korunur):

Metot	Yol	Açıklama
GET	/api/panel/stats	Toplam alarm, çözülmemiş alarm, bugünkü düşme, toplam sensör kaydı, toplam kullanıcı
GET	/api/panel/recent-alarms	Son 10 alarm (çözülmemiş öncelikli, SensorData populate)
GET	/api/panel/sensor-chart	Sensör zaman serisi verisi (hours, deviceId, userId filtreli)

GET	/api/panel/devices	Cihaz listesi (aggregation: lastSeen, magnitude, fallCount, isOnline)
-----	--------------------	---

6.8 AI Mikroservisi Modülü

AI mikroservisi, Node.js backend'den bağımsız olarak çalışan bir Python/FastAPI uygulamasıdır. Uvicorn ASGI sunucusu üzerinde <http://127.0.0.1:8000> adresinde hizmet verir. Modüler yapısı sayesinde model güncellemeleri backend'i etkilemeden gerçekleştirilebilir.

API Uç Noktaları:

Metot	Yol	Açıklama
GET	/health	Servis sağlık kontrolü (model yükleme durumu, eşik değeri)
POST	/predict	Düşme tahmin isteği (SensorWindowPayload → PredictResponse)

İstek/Yanıt Şeması:

İstek (SensorWindowPayload):

```
{
  "userId": "string",
  "deviceId": "string",
  "windowStart": "ISO 8601",
  "windowEnd": "ISO 8601",
  "sampleRateHz": 50.0,
  "readings": [{ "timestamp": "...", "accelerometer": {"x","y","z"}, "gyroscope": {"x","y","z"} }],
  "profile": "yasli" | null
}
```

Yanıt (PredictResponse):

```
{
  "prediction": 0 | 1,
  "probability": 0.0 - 1.0,
  "is_fall": true | false,
  "timestamp": "ISO 8601 UTC"
}
```

Hata Dayanıklılığı:

- Model yüklenemezse güvenli varsayılan döner (prediction: 0, probability: 0.0).
- Tahmin sırasında hata olursa aynı güvenli varsayılan döner.
- Health endpoint'i, model yüklenemediğinde "degraded" durumu raporlar.

7. API Dokümantasyonu

Tüm API uç noktaları /api öneki ile erişilebilmektedir. JWT gerektiren uç noktalarda Authorization: Bearer <token> başlığı zorunludur.

7.1 Kimlik Doğrulama (/api/auth)

Metot	Yol	Açıklama	Kimlik Doğrulama
POST	/api/auth/register	Yeni kullanıcı kaydı	✗
POST	/api/auth/login	Giriş yapma ve token alma	✗
GET	/api/auth/me	Oturum açmış kullanıcı bilgisi	✓
PUT	/api/auth/profile	Profil güncelleme (profileType, sleepSchedule, emergency contact)	✓

7.2 Sensör Verisi (/api/sensor-data)

Metot	Yol	Açıklama	Kimlik Doğrulama
POST	/api/sensor-data	Yeni sensör verisi oluşturma	✓
GET	/api/sensor-data	Sensör verisi listeleme (sayfalı, filtreli)	✓
GET	/api/sensor-data/latest	En son sensör verisi	✓
GET	/api/sensor-data/falls	Düşme tespit edilen veriler	✓

Desteklenen sorgu parametreleri: page, limit, deviceId, isFallDetected, startDate, endDate

7.3 Alarm Yönetimi (/api/alarms)

Metot	Yol	Açıklama	Kimlik Doğrulama
-------	-----	----------	------------------

GET	/api/alarms	Kullanıcının alarm listesi	✓
PATCH	/api/alarms/:id/resolve	Alarmı çözüldü olarak işaretle	✓

7.4 Admin Uç Noktaları (/api/admin)

Metot	Yol	Açıklama	Yetkilendirme
GET	/api/admin/users	Tüm kullanıcıları listele	Admin
GET	/api/admin/falls	Tüm düşme kayıtları (sayfalı)	Admin
GET	/api/admin/dashboard	Dashboard özet istatistikleri	Admin
GET	/api/admin/example	Admin erişim testi	Admin

7.5 Web Panel Uç Noktaları (/api/panel)

Metot	Yol	Açıklama	Yetkilendirme
GET	/api/panel/stats	Panel istatistik kartları	Admin
GET	/api/panel/recent-alarms	Son 10 alarm (populate'li)	Admin
GET	/api/panel/sensor-chart	Sensör zaman serisi (1–24 saat)	Admin
GET	/api/panel/devices	Cihaz listesi (aggregation)	Admin

sensor-chart sorgu parametreleri: hours (1–24), deviceId, userId

7.6 AI Servisi Uç Noktaları (Port 8000)

Metot	Yol	Açıklama
GET	/health	Servis sağlık kontrolü
POST	/predict	Düşme tahmini (SensorWindowPayload → PredictResponse)

7.7 Sağlık Kontrolü

Metot	Yol	Açıklama
GET	/health	Backend sağlık kontrolü
GET	/api/health	Backend API sağlık kontrolü

8. Test Süreci

Projenin kalite güvencesi, Jest test çerçevesi, Supertest HTTP test kütüphanesi ve bellekte çalışan izole bir MongoDB sunucusu (mongodb-memory-server) üzerine kurulu kapsamlı bir otomatik test altyapısıyla sağlanmıştır. Her test süiti, üretim veritabanından bağımsız olarak temiz bir ortamda çalıştırılmaktadır.

8.1 Test Sonuçları

Metrik	Beklenen	Gerçekleşen
Test Süiti Sayısı	8	8 / 8 geçti
Toplam Test Sayısı	65	65 / 65 geçti
Başarısız Test	0	0 (tüm testler geçti)

8.2 Kod Kapsama (Coverage) Raporu

Kapsama raporu aşağıdaki komut ile üretilmiştir: `npx jest --runInBand --coverage`

Kapsam Türü	Oran	Hedef	Durum
Statements (İfadeler)	91.52%	≥ %90	Geçti
Branches (Dallar)	83.67%	≥ %80	Geçti
Functions (Fonksiyonlar)	91.89%	≥ %90	Geçti
Lines (Satırlar)	91.52%	≥ %90	Geçti

8.3 Test Edilen Başlıklar

- Kullanıcı kayıt ve giriş akışları; geçersiz girdi validasyonları
- JWT korumalı uç noktalar; geçersiz ve süresi dolmuş token senaryoları
- Admin rol kontrolü ve yetkisiz erişim engelleme
- Sensör verisi oluşturma, listeleme, filtreleme ve sayfalama
- Düşme tespiti: eşik altı ve üstü senaryolar
- Alarm otomatik oluşturma; listeleme ve çözme
- Kullanıcı profil güncelleme
- Admin dashboard uç noktaları
- Health check uç noktaları
- Socket.IO JWT kimlik doğrulaması
- make-admin yardımcı script

9. Karşılaşılan Kısıtlar ve Geliştirme Önerileri

9.1 Mevcut Kısıtlar

- Analysis/ Modülleri: inactivityDetection.js, riskAnalyzer.js ve locationStability.js dosyaları henüz içerik barındırmamaktadır. İlgili işlevler services/ katmanında (inactivityManager.js, sensorAnalyzer.js) ve soket olay yöneticisinde doğrudan hayata geçirilmiş olup analysis/ dizinindeki dosyalar ileride yeniden yapılandırma (refactoring) sürecinde modüler biçimde doldurulabilir.
- Alert.js ve Device.js Modelleri: models/ dizinindeki Alert.js ve Device.js dosyaları içerik barındırmamaktadır. Alarm işlevselliği Alarm.js modeli aracılığıyla karşılanmakta, cihaz bilgileri ise SensorData toplamasıyla (aggregation) dinamik olarak elde edilmektedir.
- **Push Bildirim / SMS Entegrasyonu:** Acil durum kişilerine otomatik bildirim (Firebase Cloud Messaging, Twilio vb.) henüz entegre edilmemiştir. Sistem şu an yalnızca uygulama içi Socket.IO bildirimi sunmaktadır.
- **AI Modeli Tipi:** Kullanılan scikit-learn modeli Random Forest modeli olup, deep learning tabanlı bir modele (LSTM, CNN gibi zaman serisi modelleri) kıyasla temporal pattern yakalama kapasitesi sınırlı olabilir.
- **Test Kapsamı:** Web paneli ve AI mikroservisi için henüz otomatik test yazılmamıştır. Backend testleri kapsamlıdır ancak yeni eklenen panel endpoint'leri test kapsamına alınabilir.
- **Çoklu Cihaz Desteği:** Sistem teknik olarak çoklu cihazı desteklemektedir (deviceId bazlı), ancak mobil uygulamada cihaz yönetim arayüzü sınırlıdır.

9.2 Gelecek Geliştirme Önerileri

- RiskAnalyzer.js modülünün hareketsizlik süresi, düşme sıklığı ve gece aktivitesi gibi faktörleri birleştiren çok boyutlu bir risk skoru üretecek şekilde geliştirilmesi
- Push notification veya SMS entegrasyonu (Firebase Cloud Messaging ya da Twilio) aracılığıyla acil durum kişilerine otomatik bildirim
- Derin öğrenme modeli (LSTM/CNN) ile zaman serisi tabanlı düşme tespiti denenmesi
- Web paneli ve AI mikroservisi için otomatik test süreçlerinin eklenmesi
- analysis/ dizinindeki boş dosyaların refactoring ile doldurulması (services katmanından modüler ayrıştırma)
- Alert.js ve Device.js modellerinin aktif kullanıma alınması (cihaz kayıt ve takip sistemi)
- Konum verisi entegrasyonu ile locationStability.js modülünün tamamlanması

10. Sonuç

Bu çalışma kapsamında, akıllı telefon sensörlerini IoT uç düğümü olarak kullanan gerçek zamanlı bir düşme ve hareketsizlik tespiti platformu tasarlanmış ve hayata geçirilmiştir.

Proje, başlangıçtaki kural tabanlı düşme tespiti yaklaşımının ötesine geçerek makine öğrenmesi destekli çok katmanlı bir doğrulama mimarisine ulaşmıştır:

Tamamlanan İleri Düzey Özellikler:

- **AI Entegrasyonu:** Python/FastAPI tabanlı bağımsız bir makine öğrenmesi mikroservisi geliştirilmiş; scikit-learn ile eğitilmiş bir sınıflandırma modeli, ivmeölçer ve jiroskop verilerinden çıkarılan 6 istatistiksel öznitelik üzerinden düşme tahmini gerçekleştirmektedir. Model %85 olasılık eşiği ile çalışmakta, erişilemezlik durumunda kural tabanlı fallback otomatik devreye girmektedir.
- **Jiroskop Entegrasyonu:** Jiroskop verisi artık aktif olarak kullanılmakta; rad/s → °/s dönüşümü ile AI modeline girdi olarak beslenmekte ve web panelinde ayrı bir grafik serisi olarak görselleştirilmektedir.
- **Çok Aşamalı Düşme Doğrulaması:** Redis tabanlı durum makinesi (NORMAL → IMPACT_DETECTED → FALL_CONFIRMED) ile AI tespiti sonrası varyans analizi yapılarak yanlış pozitifler önemli ölçüde azaltılmıştır.
- **Hareketsizlik Tespiti:** Kullanıcının uyku takvimine göre gece/gündüz dinamik eşiklerle çalışan, PRE_ALARM geri sayımı ve kullanıcı onayı mekanizmasına sahip tam işlevsel bir hareketsizlik izleme sistemi geliştirilmiştir.
- **Web Yönetim Paneli:** React + Vite + Tailwind CSS ile gerçek zamanlı admin dashboard geliştirilmiş; Recharts ile sensör verisi görselleştirme, cihaz izleme, alarm yönetimi ve istatistik kartları içeren kapsamlı bir yönetim arayüzü sunulmaktadır.
- **Redis Durum Yönetimi:** Düşme durum makinesi, hareketsizlik durum makinesi ve sensör buffer'ı için Redis kullanılarak TTL tabanlı otomatik temizleme ve atomik durum geçişleri sağlanmıştır.
- **JWT Kalıcılığı:** Token, mobil uygulamada AsyncStorage ile kalıcı olarak saklanmakta; ortam değişkenleri .env dosyası ile yönetilmektedir.

Backend tarafında geliştirilen 65 otomatik test başarıyla geçmekte; genel kod kapsama oranı %91'in üzerinde seyretmektedir. Platform, beş bileşenli (mobil, backend, AI servisi, web panel, veri katmanı) modüler mimarisi sayesinde genişlemeye açık bir temel sunmaktadır.